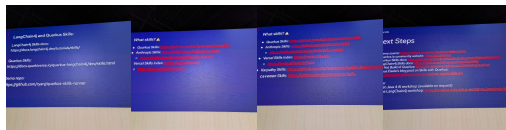


Note

Wed, May 13, 2026 12:26PM 20:44

SUMMARY KEYWORDS

Agentic skills, Quarkus, AI-assisted development, context limitations, skill portability, team collaboration, skill directory, YAML specification, Langchain for J, Java functions, tool guardrails, data compression, shell skills, multi-agent routing, skill creator.



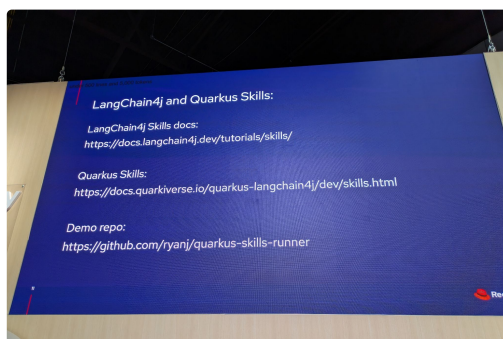
SPEAKERS

Speaker 2, Speaker 3, Speaker 4, Speaker 1

Speaker 1 00:00

But we're going to talk about agentic skills and how you can use them in general, but also specifically with quarkus. So what do we all want? Right? We all want better tools and fewer problems, right? Is AI the answer? Well, I possibly, possibly it gets us couple steps in the right direction. Let's see what skills can do to help make AI assisted development easier. So often this is there's many issues that come with generative AI, many different areas you can try to look at. One that I wanted to focus on, particularly for this talk, is context limitations, skills. Particularly, can help out with maximizing your use of your context window. So often, what people might do is come up with a prompt, run it through, find out they don't get not quite what they were looking for. So they add more detail, and then they get maybe something a little bit closer, and then they add more detail, right? And then they start pulling in all kinds of source files. Pretty soon they overload their context window and detail and requirements get lost, and then you end up with sloppy output, right? So here's a Yeah, context, overflow is basically the situation. The more detailed specifications you put in, if they overflow that window, you're just going to get mistakes out the other side, or at a minimum loss of detail in your initial specifications that you started with, right? So agent skills was introduced by Anthropic very, very short while ago, in December. You can find out a lot more information on agent skills. So, why would you want to look into skills like I said, there's there's the context improvements, but also reusability for workflows across multiple conversations, consistency. You can apply the same approach for specific tasks. You can create specialized tasks. You could say, we'll get into what particular types of specialization we have. Some examples flexibility. You can include additional files, like checklists, templates, other reference materials, maybe a style guide, other things like that can all be bundled in with the skill portability skills use an open spec that are supported by many, many different agentic services. Quarkus is just one agentic or Lang chain for Java is one agentic service. But you could also use Python. You could use Jupyter code. You could use many, many different IDE options to run these skills as well, also team collaboration, right? Once you nail one of these skills, you might want to have the rest of

your team share the same test suite or style guide or other other things, right? So a lot, a lot of good reasons to adopt skills. If you want to know about the Base Specification, it's actually really, really simple. There's more information at this URL, but at its essence, each skill is going to live in a directory containing a skill MD file. That file is going to have a little bit of YAML at the top. There's two requirements in that yaml, a name and a description. The description is going to be used by the LLM to try to find out when should this skill be used. So your description needs to be if it's if you're going to do something with testing, it better mention testing in the description right. And then below that, you're going to have markdown, ideally fewer than 500 lines. You don't want this huge, massive prompt in there. Just give it something concise. And then if you need more information, move it out into those supporting files, template files and other things that you include with the skill. All of that makes sense so far. Here's a quick example of a poem writing skill. You can give it a couple requirements, that's it, and you're off and running with now you can now you can ask your LLM or your agent, please compose me a poem. The way this is going to work is in this progressive disclosure kind of process. So the first step is a discovery Step. When the agent boots up. Quarkus, in this case, is going to send it the name and description, just that YAML chunk from each of the skills, so the LLM will know when it needs to ask for the rest of the skill information. This is going to help keep that context really lean, because you're not even loading the whole prompt yet right. Next step is activation when a task matches that skill description, the agent will send, or really the Yeah, corpus, in this case, is going to send the full skill MD instructions and related files into the LLM context right next you have execution. At that point, the agent follows the instructions, optionally executing any bundled code or loaded loading additional reference files as needed. So that's kind of the last step of the process skills in Java. You're going to be using Lang chain for J for any kind of Java agentic work. Quarkus has excellent support for Lang chain for Jay, if you wanted the upstream Lang chain, Lang chain skills docs, you can find it at that top link. This quarkus skills are here at the second link, and I have some demo sources available at this third link. Down here, we'll be able to pull off my demo over the shoulder here. I'll try it out. Okay, so if you wanted to use the quarkus line chain for J skills extension in one of your existing Java apps, all you need to do is add the dependency your palm xml, configure your application properties file to tell it where these skills are going to be referenced from. I'm using an example where it's trying to read from my local directory. That's what most IDEs do. And I wanted to test some skills that I borrowed from Claude. So that's what I set up. You could also refer to your class path and have skills that are kind of bundled inside the jar. That might be a better way if you're rather than having it in the repo folder right longer term. And then you just add an annotation to your assistant resource or your assistant object and add this that'll populate the system message with that initial YAML that we had all the name and description, all right, demo. Let me see if I can. That's not what I wanted. Let me see if I can load a





S

Speaker 2 07:30

demo so I have, okay, I've got

S

Speaker 3 07:40

halfway through my time bump

S

Speaker 2 07:45

the bunt here and make this full screen. Oh yeah, yeah, that might be easier. Okay, all right, okay, so All right,

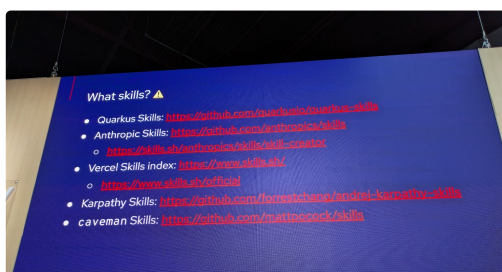
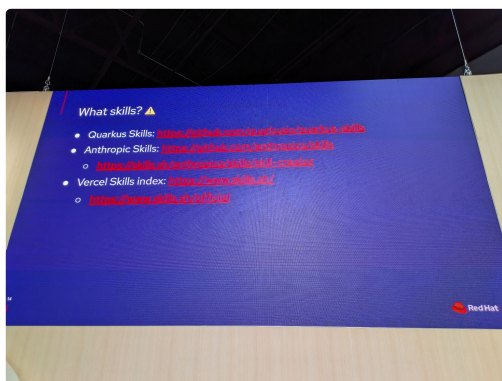
S

Speaker 1 08:02

and I'll pull over one more deal. We can take a quick look at this first so here's that minimal poem writing example. I have a couple other examples in the repo. Here's something showing a multi step example generating API docs. You can have multiple steps to run things in sequence in here, this one also references to external files. Here's a input processing example, where we roll some dice and based on the input, we're going to possibly run a shell command right. This may or may not be supported in your agent based on the availability of different tools. Right Claude code is automatically going to have a tool built into their agent that provides shell access. Quarkus, you have to manually kind of enable that functionality. It's in a kind of experimental feature. If I have time, I'll cover that, but that's included in my demo script further down. Let's see. So if you wanted to integrate really tightly with your existing Java stack, you could do a skill, something like this, where you have a process order skill, right? And this is maybe going to call direct Java functions that you have exposed as tools, right? So this is totally possible. You can't run this in Claude code. It's no longer portable to other skill environments, but the concept of skills is portable, and you can use it when running from a Java agent to make calls into your tools that were exposed via Java. So excellent kind of bonus there. Also just be careful if you're providing access to run shell commands or if you're borrowing scripts from around the internet, committing those scripts to your repo is an essential endorsement to your team members. So read before you commit, read before you use, read it twice before you add it to your repo, because you don't want to have skills end up used for data exfiltration or some other kind of malicious activity, so be careful when you use skills. Let me get into my demo here, so I have let me see if I can copy this command and fire it off. Is about right? Okay, and then I'm going to ask it a basic question, what skills and tools do you have access to? I just made a very, very simple quarkus agent. This is not using a web interface. I just wanted to make a simple CLI interface so here it recognized it already has a couple different skills that were inside the skills directory that I identified, and it's also risking listing some various tools that I went ahead and added in advance. I gave it the ability to check the current time. I gave it the ability to do basic file reads and writes within my repo. Quarkus does not do that by default. I mocked out additional tooling so that I could try out skills from Claude or skills from other skill

providers that weren't necessarily from a Java stack. So that's why I have a couple extra tools identified in there. So let's see if our poem writing skill gets triggered based on the description we provided here. It says, let's write a poem about an optimistic tree. I think that's what we had in the description, or maybe I said about a red hat. Anyway, this is the success. This is the success case, we got our poem that meets the requirements as defined in the in the in the example, if you wanted to be more explicit, if that description did not get triggered for some reason, you could say, Hey, LLM use tools to call. You could do this. You could do this, Hey, activate the poem writing skill and use it to compose a new poem that's a little bit more explicit. Doesn't need to rely on your description as heavily, right? May increase your accuracy with hitting that particular skill. You could be even more explicit. You could use function calling to run the Activate Skill tool that quarkus has provided, and then say, go use this specific tool by name, right? Rather than relying on the description, let's just rely on the name and tool calling. So you could get very explicit, if you wanted to all of those examples work. Let's try our dice rolling example. So we're going to refer to that skill by name in order to make sure it gets run. I have some I'm not sure why it's I put some little debug messages here so I could tell what the LLM is doing. It looks like it's trying to list a bunch of files. For some reason it's writing files. I need to be more cautious about the tool access I grant, right? But eventually it rolled a D 12 and got a result of five. I'm not sure what it was doing with all this file access, but I'm glad I added some additional logging support so I can try to try to diagnose that I'm using a very minimal model. I'm using GPT four, oh mini I wanted to pick the smallest, least effective model so I could test edge cases. And so this is definitely an edge case. All those looking up different files is, I don't know. It may depend on the model you choose. Let's try another one. This one is going to make some function calls using those read file functions that I mocked out so this should, whoops, did I paste the whole thing? That's not good. I didn't mean to paste that. I meant to do this. One. All right, let's try it again. Init repo here, I used a slash command. I used slash init. Whoops. All right, I'm gonna bail out and start over. Okay, I think pasting in that droplet before mess it up. All right, let me go. Let me try to do just slash init repo and see if that triggers our init repo skill that should put a hello world inside our repository, and kind of bootstrap our repo a lot of agent, agentic development systems might add, like, an agents.in D to tell it how to get started with your with working on your project. I would probably, you know, do something like that in a, you know, engineering capacity, add an agents in G that's relevant to your org, or something. All right, five minutes. Let me get to, oh, man, I so I have a great section on shell skills that I don't have time to cover today. But if you do go ahead and enable shell skills, it's pretty easy to do. There's just three lines you need to uncomment in some of my code. Once you do that, you could do things like you could install random skills from the internet. This is a skill from the quarkus team. Then we could do something like maven package. This time, since it's a shell skill, I'm going to run this inside a container, right, just for added safety. So what I do in this command is run this inside a container and then use function calling to try to test the bounds of that container. Can I check which OS is running inside the container? Can I run shell commands to check my user ID? Can I list file processes will dice rolling work by actually running that bash command, instead of simulating the bash command like it's doing right now? And can I run this nice quarkus upgrade to get me to the next version of quarkus? Those are all possible using custom skills from around the internet, possibly with need of that shell skills support. Use a sandbox if you're going to do shell skills or run it in a container. Make sure you don't have data sitting nearby, and let me get back to my wrap up. So here's some more skills you can use from all over the internet. First, this is the quarkus skill that I was going to demo today. Anthropic has a nice collection of skills. They have a skill creator skill that will help create and refine your skills over time. So that's a nice one to look into. Purcell. I don't trust this resource as much, but they have a very large collection of skills, and one that's listed as official that's maybe safer to try. I definitely want to give you warnings about the vercel. I've heard that a large portion of the skills are perhaps compromised, so just

be careful read them before you use them. There's some other really fun skills. I've heard great things about the caveman skill that encourages your agent to talk in the fewest number of words possible, that saves you tokens and perhaps gives you more budget for AI development by doing caveman speak during your development phase. Hopefully that doesn't introduce additional compression, or maybe compression in just the right areas while keeping your specs right. Compress the English language, but not the interactions, but not the tool calling and the specs right. So skills present an interesting security risk. Use them with caution. They depend on tools to interact with the world. Be careful which tools you enable. Skill portability is going to be limited by Tool Availability. Add just enough tooling for your requirements. Quarkus allows these Java functions to be easily exposed as tools, to increase capability, reliability, efficiency and tool guardrails are a great thing to add in when you're using these Java tools, since you're loading these skills directly into the context. It's going to bypass your input guardrails. So you're going to want to use tool guardrails. In addition, tool guardrails can also validate all the input coming in and the output going out as well. So really good, really good features there in the tool guardrails. So hopefully we go from potential of malicious prompts and data exfiltration if we use our tools responsibly, read our scripts first. It looks like we got some data compression on this image, but hopefully it's the right the right idea skills, ideally will enhance efficiency, reliability, accuracy and specialization via that progressive disclosure process. So are we there yet? We still have lots of work to do, but I do believe that skills are a major stepping stone towards improved context management and ability add massive new capabilities to your agent without having to write all the code yourself, right? Yeah, a lot of great links here. Also, I would love to mention Marcus Isley, my one of my team members, has a really killer blog post on Java agent skills, where he does multi agents, Agent routing, all kinds of cool stuff, also the Lang chain 4j skill workshop has great examples of doing multi agents, parallel agents, sequence agents, all of that fancy stuff that is it for me, feel free to follow up with me if you have any questions. Thank you very much. Did





S

Speaker 4 20:41

Ryan.